Predicting F1 pit stops

Scope

The goal of the project is to make a machine learning model which can make predictions on whether or not an F1 driver will pit on the next lap. Currently pit stops are predicted more on a ballpark estimate. For instance, the tire manufacturer will often estimate a particulare compound will last about 15 to 23 laps. Each team will also make calculations based on real-time race data, tire degradation models, game theory, and machine learning.

Pit stop strategies in F1 is an essential part of the race strategy. There are two main strategies called undercutting and overcutting, the former means you pit first to use fresh tires to overtake an opponent. The latter means you try to drive longer on old tires than your opponent, to gain positions. A pit stop usually takes about 30 seconds of their time, so timing it right in regard to where you end up in the field, and how you perform on new vs old tires is essential parts of predicting if a driver will pit.

That said, it's still an interesting problem to try to solve with a simpler model which doesn't have so many moving parts.

Metrics

The competition this dataset is based on rates submissions on ROC Area under Curve, so we'll mainly focus on that, as well as precision and recall. Since the dataset is weighted heavily in favor of "no it didn't pit next lap" with about 80% of all rows being

that, accuracy won't be a very useful metric. As you can see in this histogram:

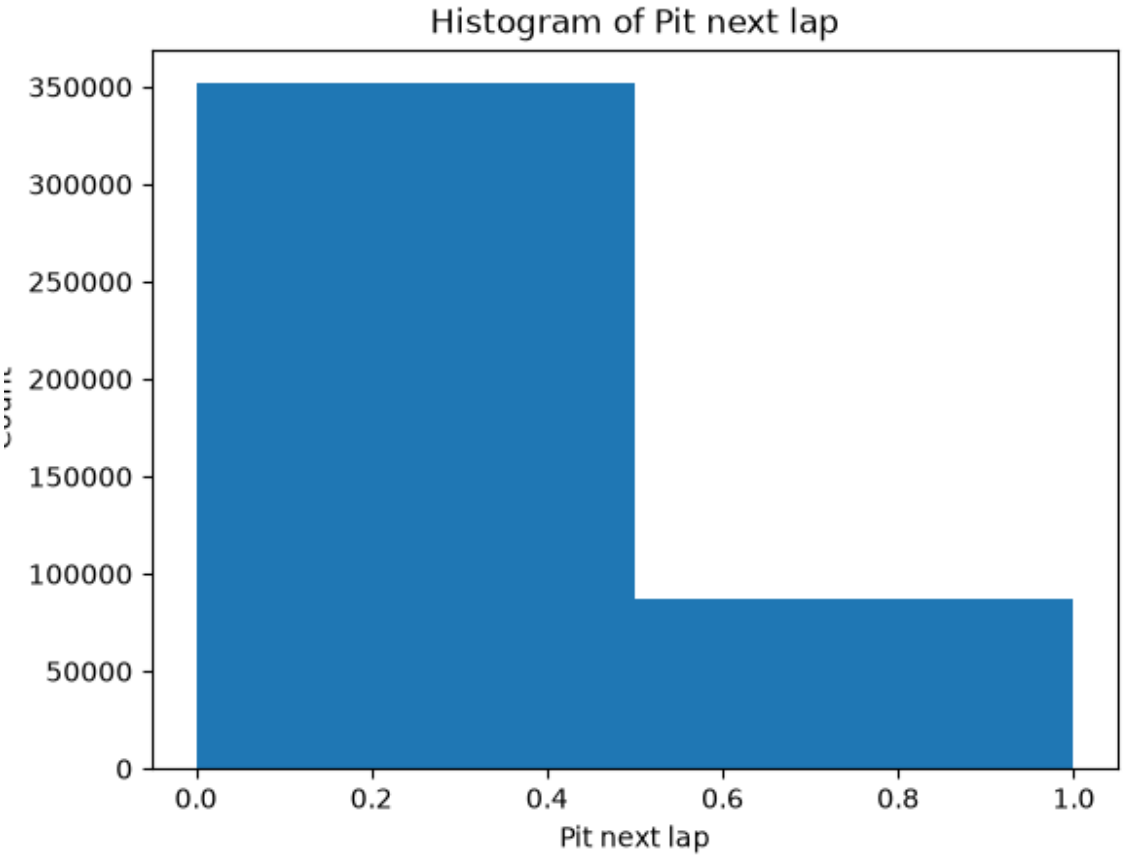


Figure 1 Histogram of Pit next lap

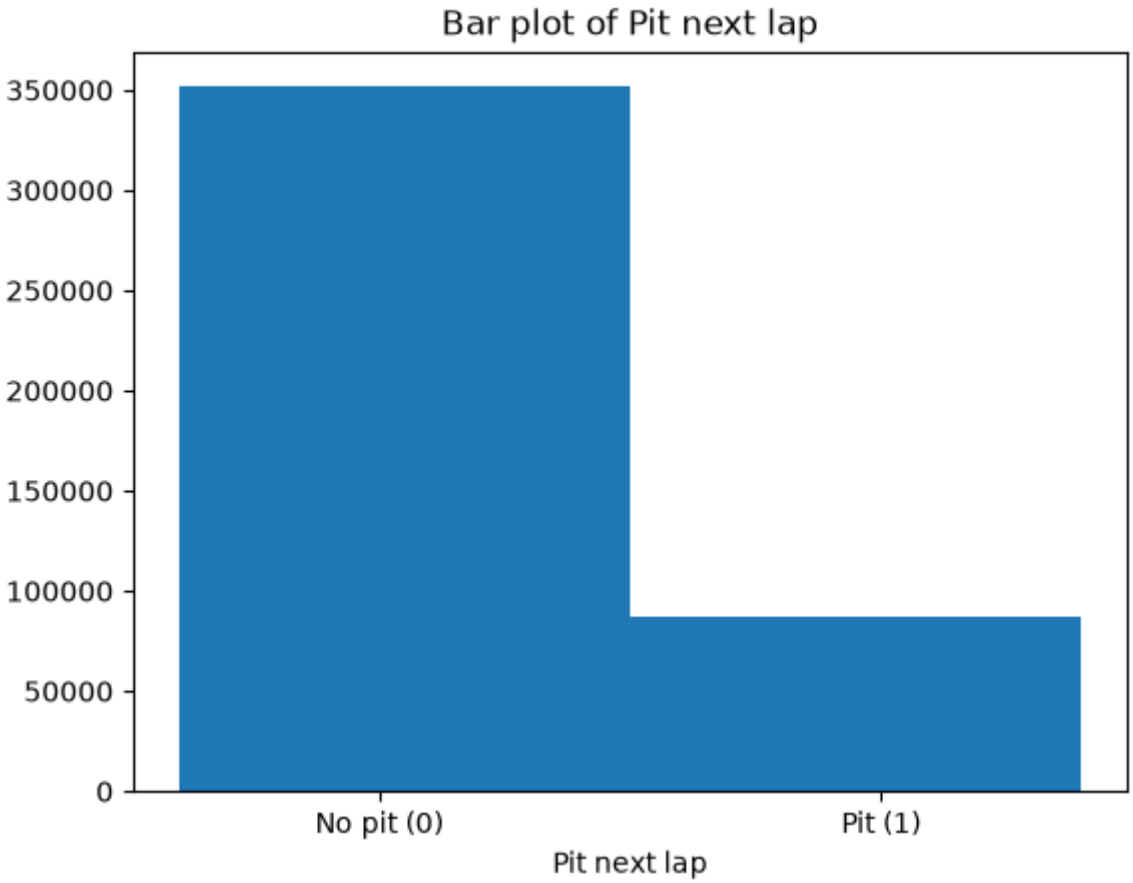


Figure 2 bar plot of pit next lap

Data

Based on our prior knowledge of F1 as fans of the sport, our theory is that the most important features will be: compound, tyre life, lap time delta, and cumulative degradation. There are some variables which could play a factor such as what stint they're on, because all races have one mandatory pit stop. Combined with years before 2024, teams usually had two pit stops, but from 2024 and beyond one stop strategies became the norm. You also have some drivers which might be slightly better at managing tyres compared to others. Which race they're driving could matter to a degree, with some races having higher temperatures which degrade tyres a bit faster. That said, it's very weather dependent, and we believe it will be harder to generalize. We initially believed position was not as relevant, since you already have position change as a feature. The thing is, every race demands you pit once, or in rare cases twice. Then you have a race like Monaco, where it's near impossible to overtake. If you're leading the race and have done your mandatory stops, you'll probably not pit again unless something extraordinary happens. The position will also tell us something else, if you're leading the

race, you're not driving in "dirty air", which will positively affect your tyres. The label PitStop represents whether the car pitted this lap, which means the car has pitted on that lap. It is extremely unlikely for a car that just pitted, to pit again the next lap. So this will be a helpful feature to check.

For those reasons we'll try making one model using race, compound, pit stop, lap number, stint, tyre life, position, lap time, lap time delta, cumulative degradation, and race progress, and one where we swap position for position change.

I represented the data in a correlation heatmap, checking the numeric columns:

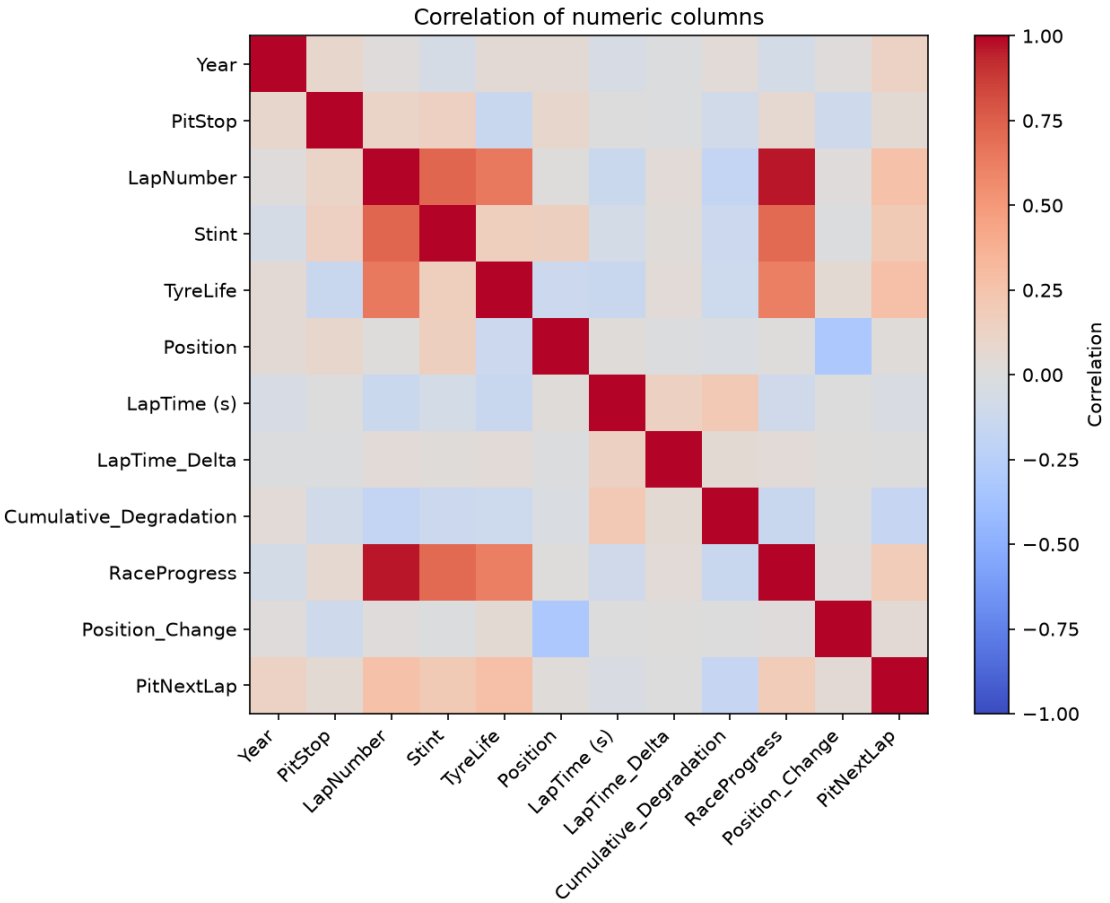


Figure 3 Correlation heatmap

The diagonal can be ignored, as it's the same value checked against itself. We also see a strong correlation between race progress and lapnumber, which makes sense, since race progress is just number of laps divided by total laps. Stint, Lap number, and race progress have a strong correlation, but aren't necessarily a redundant cluster. Later in a race you're on a higher stint, and you're tyres usually have done more laps.
The strongest values seem to be Lap number, and Tyre life at about 0.25-0.3, then Stint, and Race Progress at about 0.2 and year slightly positive. Since there's a correlation between lapnumber and race progress, we'll pick one. Thinking ahead to the website

where this model will be used, we'll pick lap numbers, as that's easier for a user to know than to calculate race progress.

Checking the laptime delta against pit next lap shows that about 4.9% of all the entries have zero value for the laptime delta. Which a large portion is the first lap of the race, when you have no other lap to compare to. Another explanation is that if you've just pitted you've no prior laps on that set to compare against.

We tested the Monaco race hypothesis, and it seems to have been disproven:

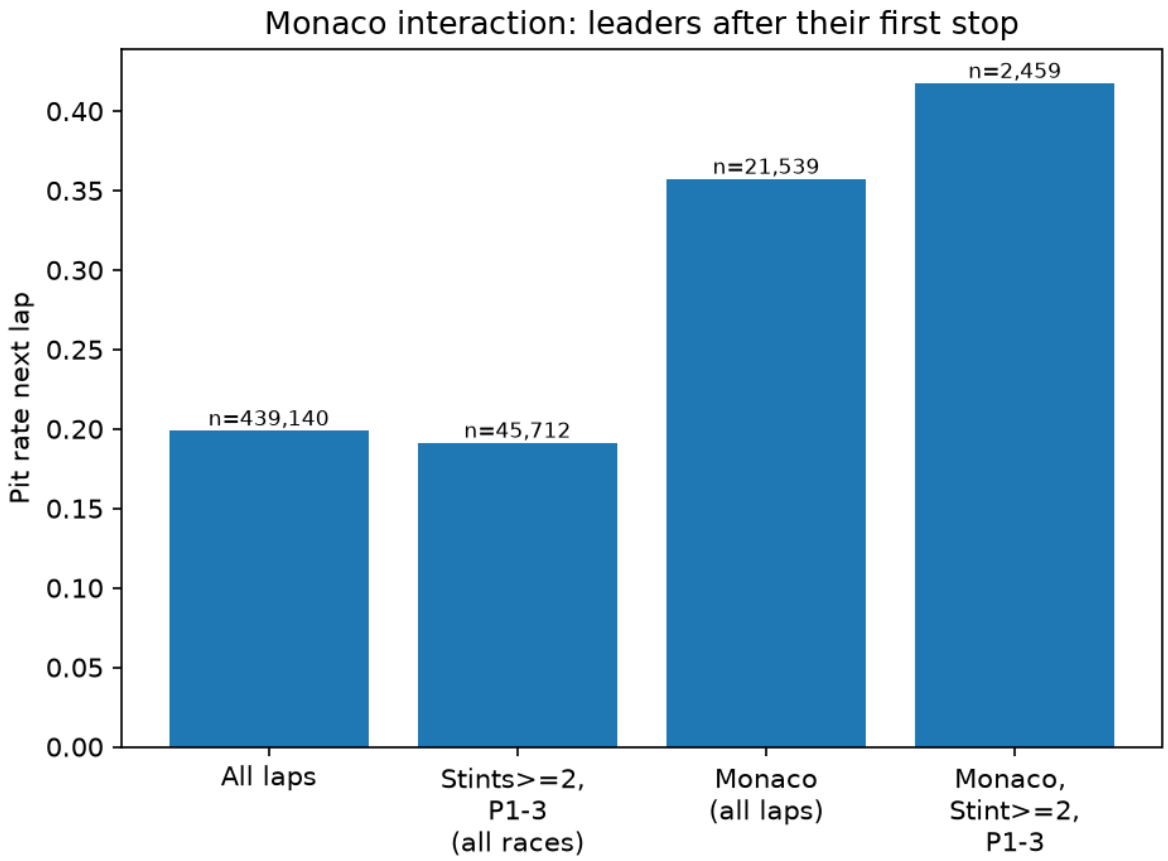


Figure 4 pit rate by race leaders in Monaco

What it does show, is that what race you're doing seems to have some impact on whether or not you pit the next lap or not. You also see that being in the top three doesn't seem to dictate whether or not you pit, it's more or less on the baseline. One thing to note here, is that the data is synthetic, so real-world intuition like "nobody pits more than needed at Monaco" might not hold in generated data. Checking the individual years, we see that this Monaco average is not consistent year by year, for instance 76% of all laps in Monaco in 2024 were followed by a pit stop, as opposed to the sub 40% for the other years. 2023 also had 0.7% pit stops on their laps, which is also odd. Which is unlikely for a real race, but a symptom of a synthetic dataset.

We also had the hypothesis that if you had pitted on this lap, you probably wouldn't pit on the next lap. The data shows that this is not true, about 25% of all laps that just had a pit, were followed by another pit stop. Which is highly unrealistic, and again, another symptom of a synthetic dataset.

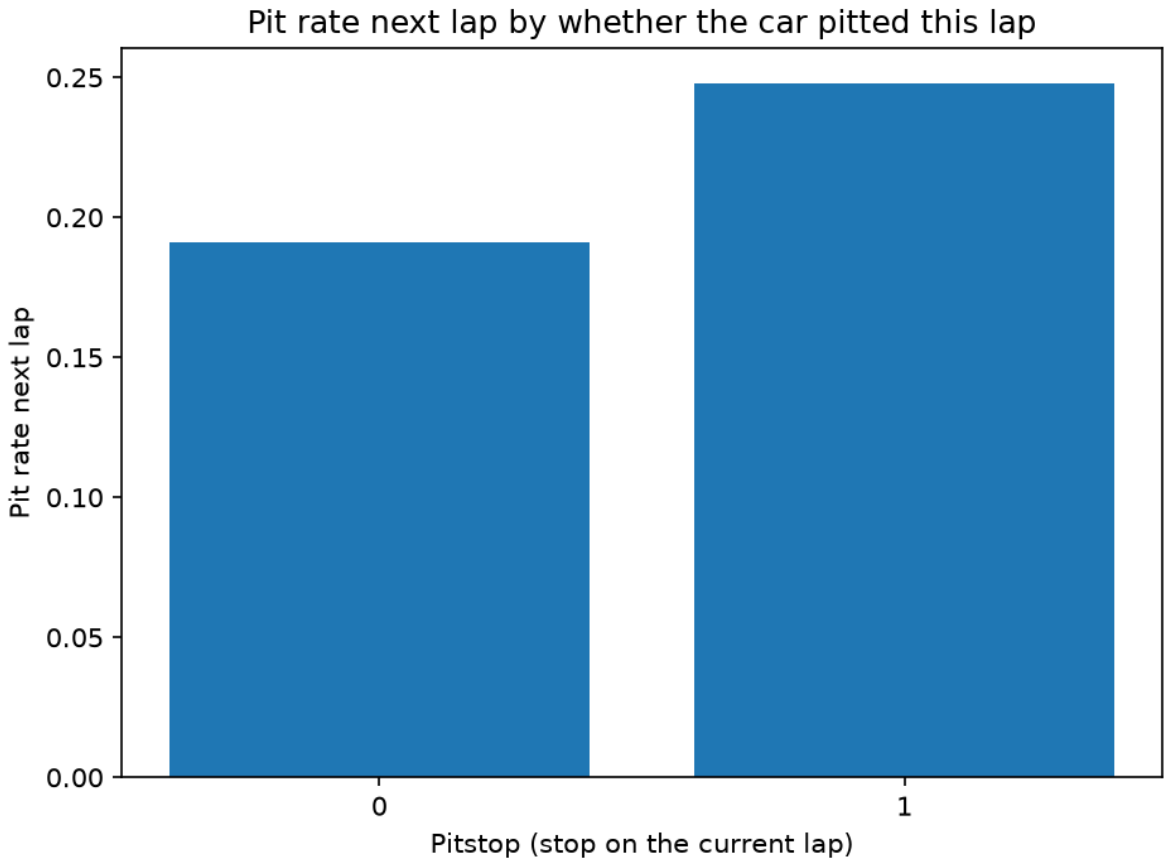


Figure 5 chart showing pit rates on laps that just had a pitstop

This shows that it's a weak signal, which can be useful in a tree based model, but we won't focus much on it beyond that.

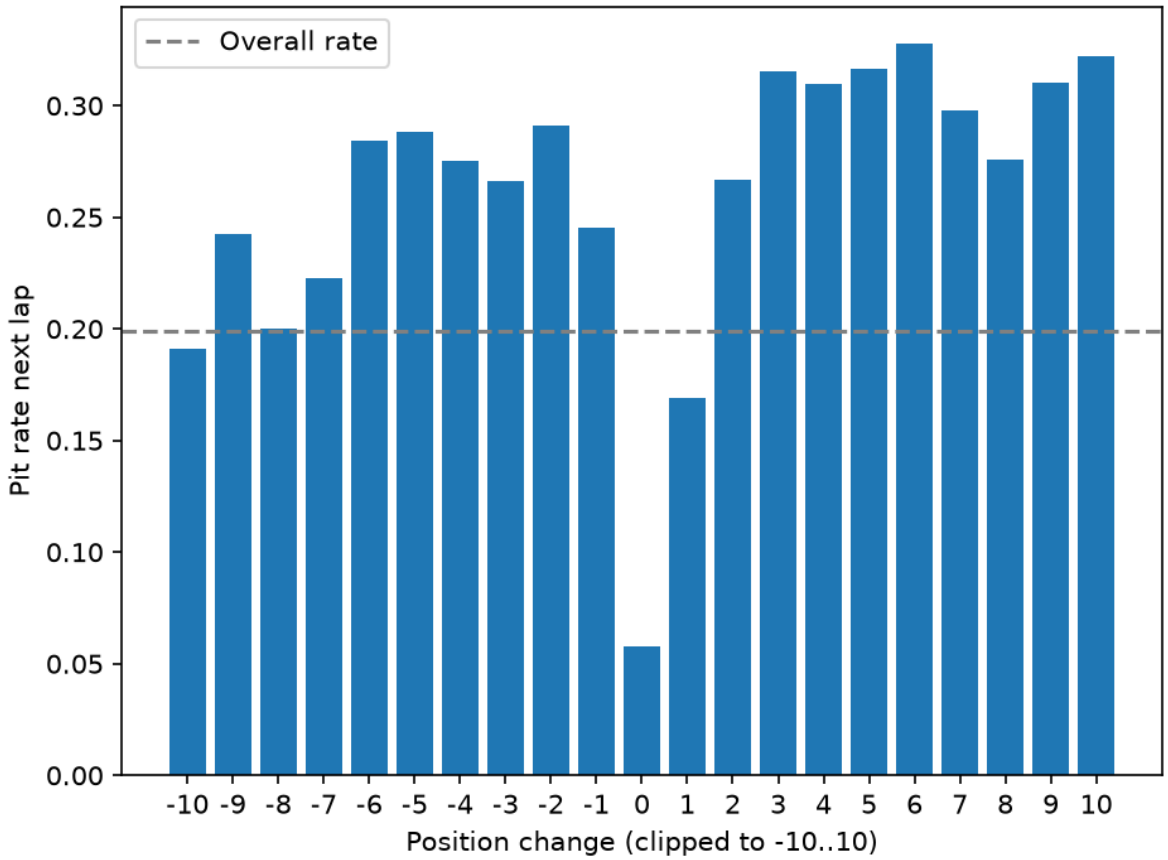


Figure 6 diagram showing pit rate in relation to change in position

No change in position seems to have the lowest pit rate. A hypothesis we had was that your change in position mattered more than the position itself, it seems that the size of the change is the more important factor. A big change could mean that a car has gained or lost positions due to having pitted, or other cars ahead pitting. Often cars on similar compounds will hit pit windows close to each other, causing a pit cycle.

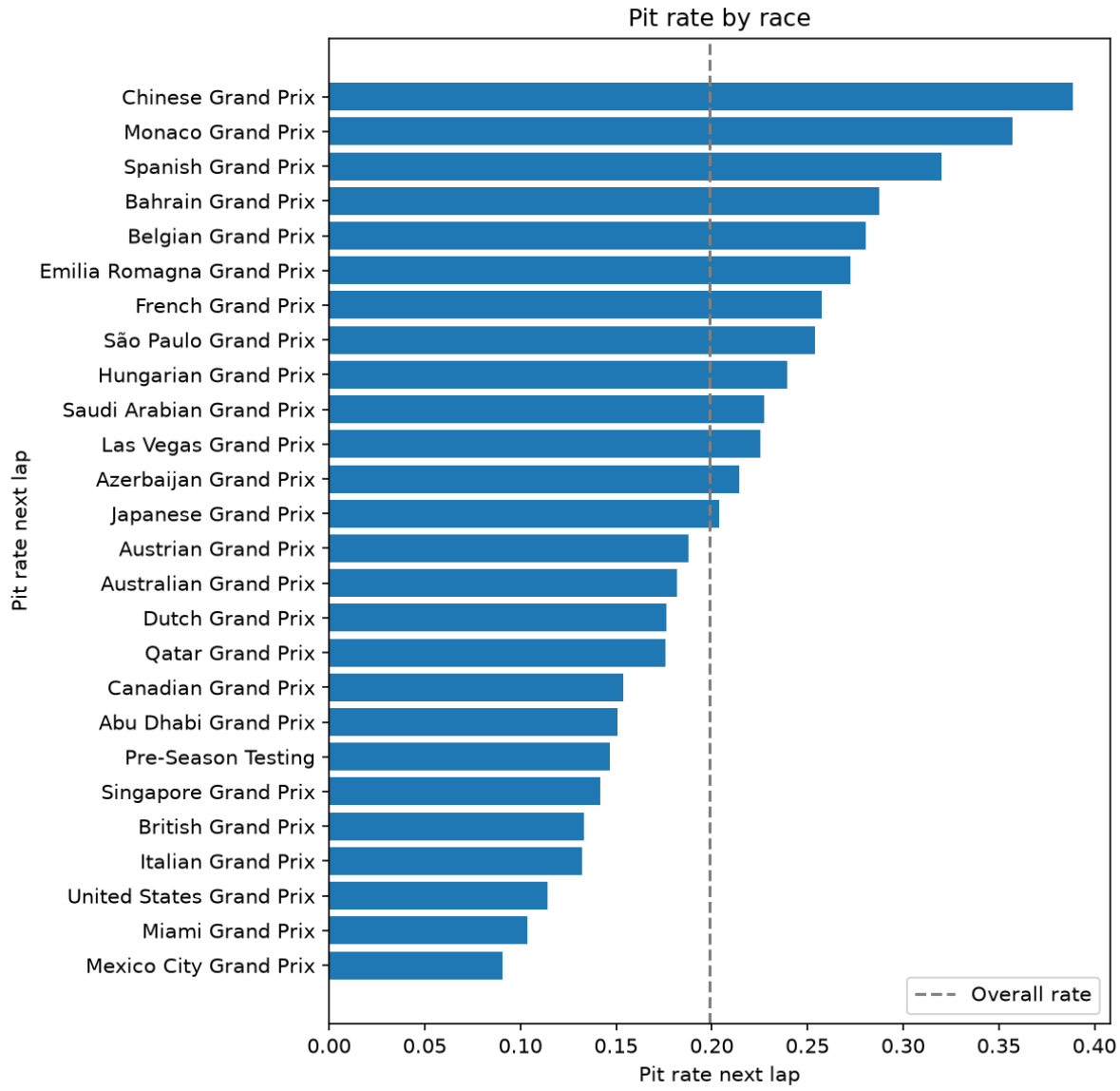


Figure 7 Pit rate by race

Here we can see that there's variety in pit stops next lap based on which race you're driving, proving it to be a relevant feature to use when training a model. As we mentioned earlier with the Monaco example, there can be some huge swings from year to year.

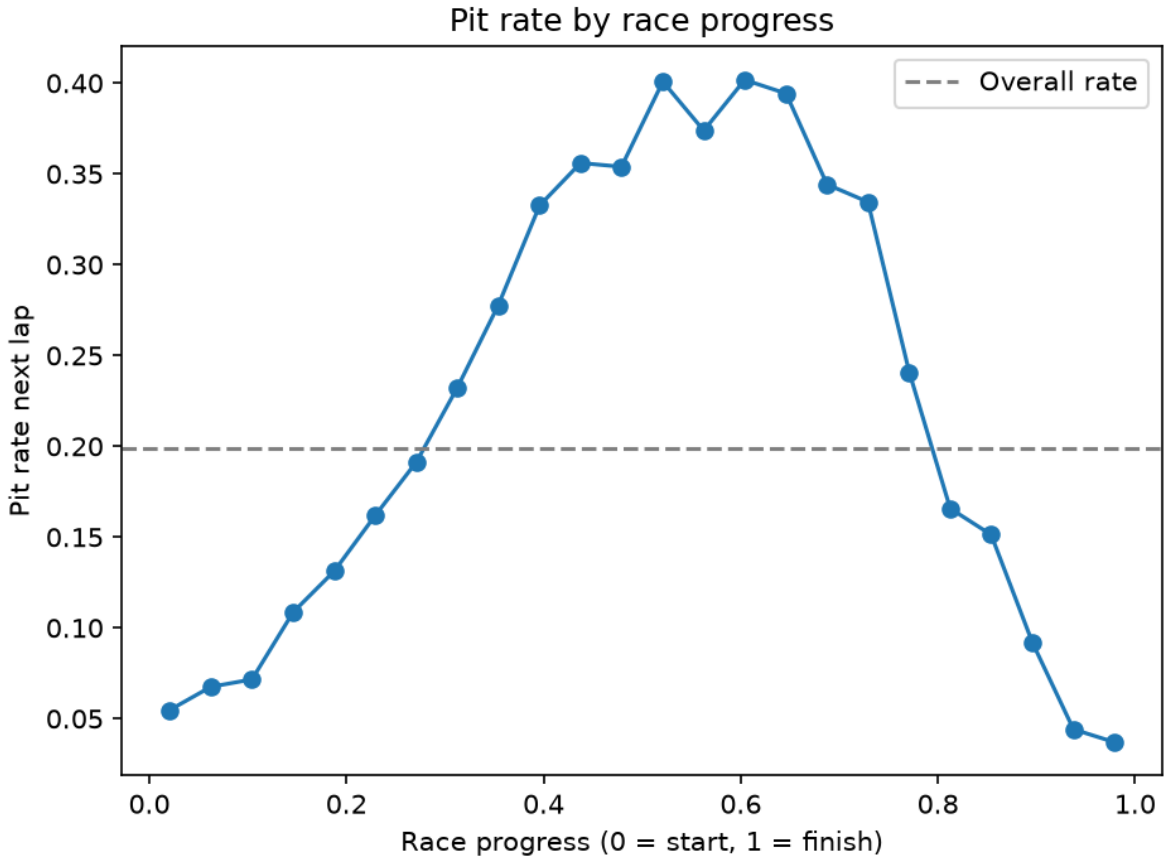


Figure 8 Pit rate by race progress

This shows that there's a clear pattern in regard to where you are in the race, and pitting. You're less likely to pit at the beginning and end of the race, but around the middle you're more likely to pit. This means we'll have to try with and without raceprogress in our model. Perhaps make one with, and one without comparing how they do, and drop it if they perform similarly.

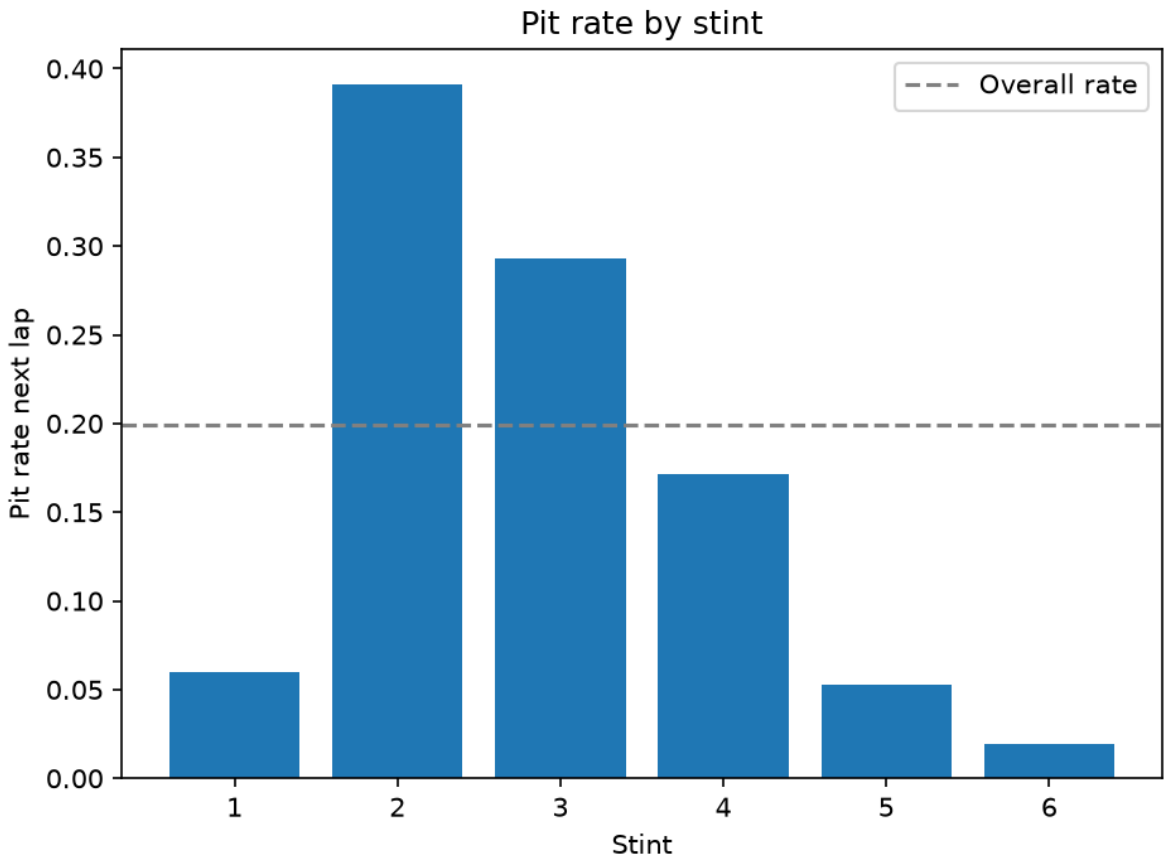


Figure 9 pit rate by stint

The data seems to indicate that if you're on your first stint, you're less likely to pit than when you're on your second, third, or fourth stint. This is unrealistic compared to real world racing, since there's a minimum of one pit stop required. It would be more sensible if stint 1 had a high rate, which declined as stints increased. This is because you do lose valuable time in pit, and preferable only pit when you need to because the tyres are too degraded. It looks like there's some correlation between stint and race

progress. We decided to check this by making a plot with stint and race progress:

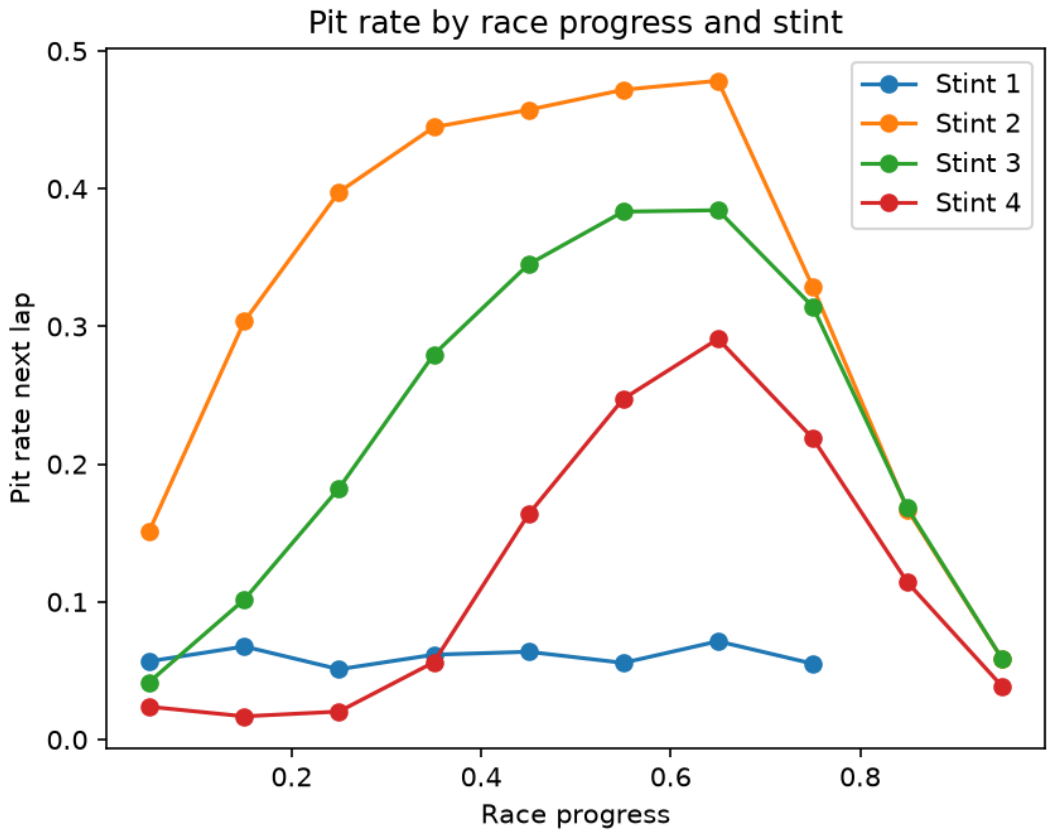


Figure 10 Pit rate by race progress and stint

Here we can see that if a car hasn't pitted yet, they have a relatively steady 5-7% chance to pit across the whole race, with stint 2, 3, and 4 having a high chance in the middle of the race. The second stint rises first, and highest, with 3 and 4 both lower and later to rise. Again the data seems to contradict real-world racing logic and consensus on this point. You should have a higher pit chance the further you go on your first stint. The latter stints pitting more mid race does make sense.

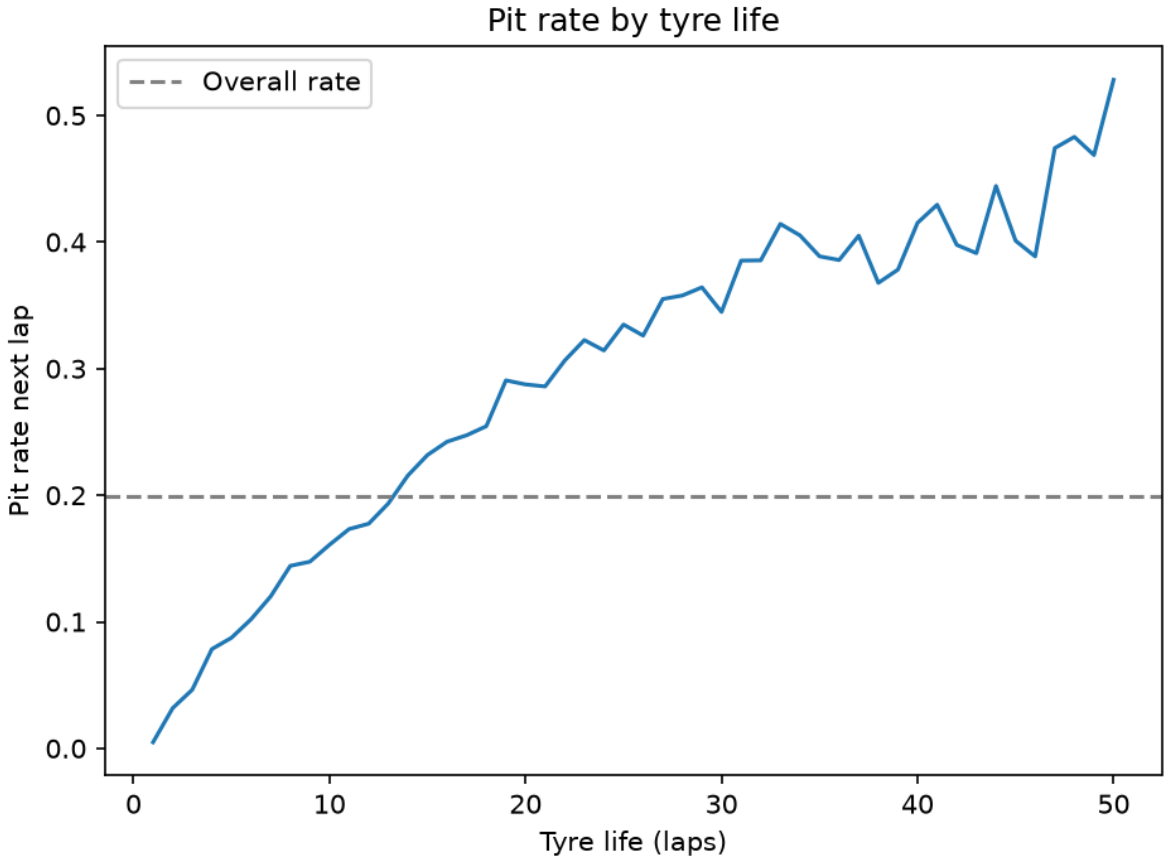


Figure 11 Pit rate by tyre life

This supports our earlier hypothesis that your chances of pitting is higher the older your tyres are, and this is one of the more logical plots in relation to real-world intuition that we've had. It indicates that Tyre life is one of the strongest single features we've found, and will be essential in training the model.

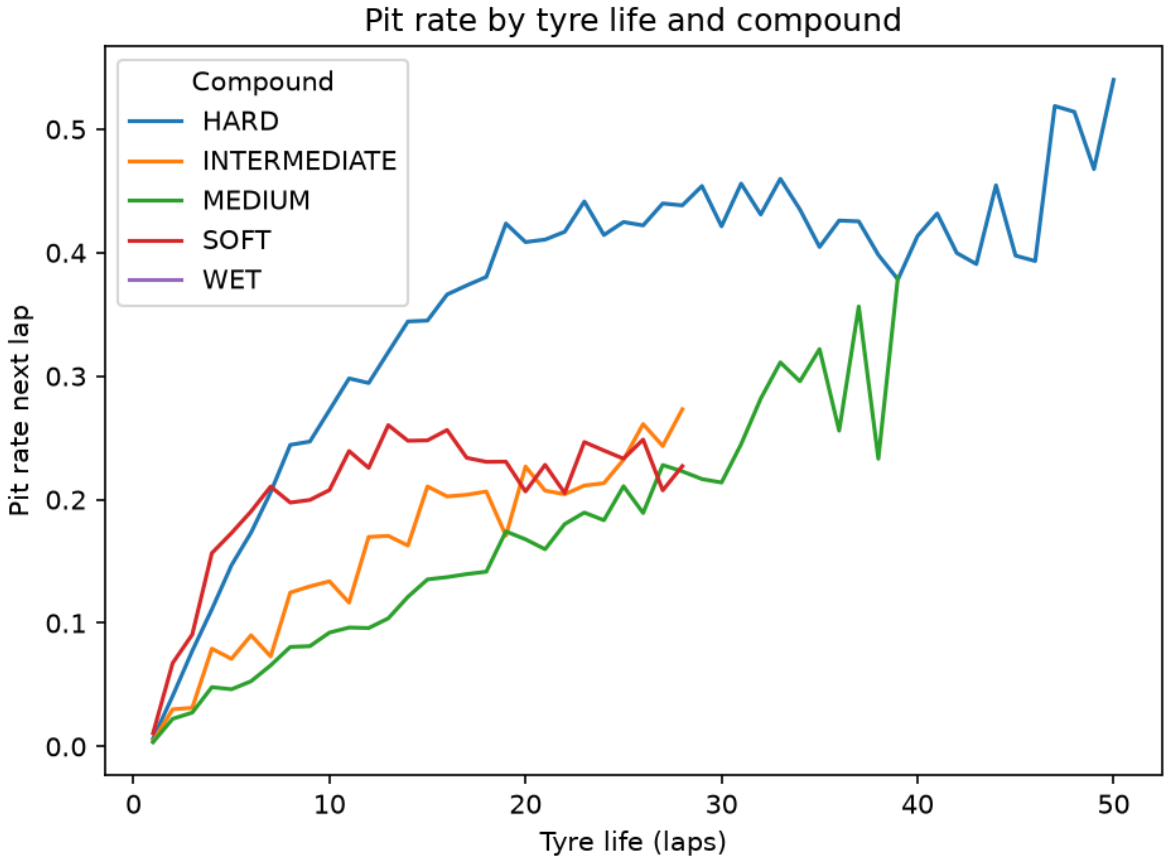


Figure 12 Pit rate by tyre life and compound

Here we can see that the various compounds behave differently in regardsto tyre life and pitting. It seems that compound usage was strongly tied to stint and race stage. A hard tyre is less likely to pit early, but as the race wears on, they're more likely to pit.

Models

Based on the data analysis it seems a tree-based model might be the best fit for this. We will explore different models and compare performances across these to figure out which might be best. We initially ran a 5-fold cross validation on the following models available in ML.NET: FastTree, LightGBM, FastForest, and LbfgsLogisticRegression. Features minus Race and Year:

Model:	FastTree	LightGBM	LbfgsLogisticRegression	FastForest
ROC AUC:	0,9323±0,0222	0,9297±0,0215	0,7384±0,0351	0,9064±0,0233
PR AUC:	0,7301±0,0576	0,7277±0,0560	0,3928±0,1077	0,6747±0,0451
Accuracy:	0,8902±0,0242	0,8864±0,0235	0,8109±0,0259	0,8768±0,0249
F1 Score:	0,6748±0,0602	0,6486±0,0584	0,3240±0,0778	0,6098±0,0214
Precision:	0,7077±0,0460	0,7107±0,0443	0,4612±0,1093	0,7005±0,0446

Recall:	0,6455±0,0747	0,6094±0,0746	0,2564±0,0766	0,5415±0,0306

As predicted, the regression based model performs poorly, while the FastTree and LightGBM models are fairly equal in performance. The FastForest model also has a significant gap in performance compared to FastTree and LightGBM, so we'll be focusing on LightGBM and FastTree models going forward.

In this check we first ran it with the pipeline unmodified, but in the second run we made a new column RaceYear, which combined year and race, and did a group split on it. So that it was less likely that the model just memorized the race and used that to estimate identical races from training to test. This change caused a slight drop in performance of the three top performers, and a larger one on the Logistic Regression model. Laps from the same race are highly correlated, so including those in both test and training causes leakage and a slightly inflated performance across all models. This grouping mimics the intended use case, where the models might face unseen races or future races. We ran this cross-validation on a few different setups, one where we removed Race from the features, and another where we removed Year, all grouped by the combined RaceYear column.

Base features and Year:

Model:	FastTree	LightGBM	LbfgsLogisticRegression	FastForest
ROC AUC:	0,9332±0,0218	0,9342±0,0217	0,7545±0,0100	0,9103±0,0274
PR AUC:	0,7220±0,0563	0,7236±0,0588	0,4016±0,0765	0,6643±0,0628
Accuracy:	0,8898±0,0244	0,8907±0,0241	0,8260±0,0274	0,8768±0,0256
F1 Score:	0,6718±0,0518	0,6718±0,0496	0,2174±0,0542	0,6164±0,0320
Precision:	0,7107±0,0511	0,7107±0,0518	0,5564±0,1186	0,6914±0,0584
Recall:	0,6387±0,0626	0,6387±0,0602	0,1390±0,0442	0,5575±0,0280

Base features and Race:

Model:	FastTree	LightGBM	LbfgsLogisticRegression	FastForest
ROC AUC:	0,9066±0,0268	0,8751±0,0517	0,7508±0,1067	0,8853±0,0266
PR AUC:	0,6868±0,0524	0,6536±0,0879	0,4425±0,1254	0,6487±0,0504
Accuracy:	0,8835±0,0260	0,8725±0,0267	0,8268±0,0417	0,8693±0,0254
F1 Score:	0,6318±0,0878	0,5873±0,1291	0,3876±0,1140	0,5499±0,0353
Precision:	0,7129±0,0573	0,6829±0,0568	0,5186±0,1395	0,7109±0,0571
Recall:	0,5786±0,1205	0,5448±01749	0,3120±0,0986	0,4509±0,0446

Base with Race and Year:

Model:	FastTree	LightGBM	LbfgsLogisticRegression	FastForest
ROC AUC:	0,9323±0,0233	0,9301±0,0235	0,7384±0,0665	0,9061±0,0279

PR AUC:	0,7301±0,0704	0,7280±0,0704	0,3928±0,1104	0,6755±0,0649
Accuracy:	0,8902±0,0263	0,8872±0,0262	0,8109±0,0552	0,8770±0,0253
F1 Score:	0,6748±0,0598	0,6544±0,0844	0,3240±0,1354	0,6059±0,0326
Precision:	0,7077±0,0587	0,7100±0,0600	0,4612±0,1354	0,7049±0,0537
Recall:	0,6455±0,0639	0,6169±0,1190	0,2564±0,0951	0,5321±0,0285

We also ran the same measurements, with the same choice of features, but, grouped by Race instead, but skipping the run on all features minus race.

Base features:

Model:	FastTree	LightGBM	LbfgsLogisticRegression	FastForest
ROC AUC:	0,9224±0,0169	0,9247±0,0150	0,8404±0,0125	0,8820±0,0205
PR AUC:	0,6951±0,0737	0,6956±0,0775	0,5168±0,0671	0,6229±0,0840
Accuracy:	0,8852±0,0307	0,8857±0,0291	0,8421±0,0406	0,8666±0,0408
F1 Score:	0,6506±0,0424	0,6506±0,0455	0,4151±0,0486	0,5407±0,0801
Precision:	0,6954±0,0674	0,6954±0,0674	0,6053±0,0860	0,6808±0,0834
Recall:	0,6145±0,0481	0,6145±0,0481	0,3245±0,0724	0,4542±0,0861

Base features and Year:

Model:	FastTree	LightGBM	LbfgsLogisticRegression	FastForest
ROC AUC:	0,9312±0,0118	0,9321±0,0123	0,7762±0,0461	0,9074±0,0176
PR AUC:	0,7096±0,0636	0,7088±0,0623	0,4249±0,1142	0,6467±0,0801
Accuracy:	0,8869±0,0301	0,8865±0,0294	0,8287±0,0470	0,8752±0,0370
F1 Score:	0,6602±0,0342	0,6610±0,0340	0,2527±0,1292	0,6017±0,0807
Precision:	0,6970±0,0622	0,6930±0,0603	0,5438±0,1627	0,6736±0,0768
Recall:	0,6304±0,0431	0,6351±0,0443	0,1768±0,1122	0,5486±0,0942

A common theme among all these runs was that FastTree and LightGBM performed very similarly, with LightGBM getting a slight edge. Another theme was that both FastForest, and Logistic Regression performer worse, with the latter often being 0.1-0.2 points below the others. In regard to training time LightGBM took the shortest of the best performers with 5 seconds compared to FastTree's 19 seconds. For that reason, and their equal performance, we decided to move forward with the LightGBM model. One note was that Year seems to help a little, while Race under the RaceYear grouping seems to hurt the performance. With LightGBM for instance dropping from around 0.925 to 0.875, and seems to overfit to circuit-specific behavior. Initially we thought Race as a feature would help with predictions, but the data points in the other direction. Race will be dropped in future tests, and we'll continue grouping by RaceYear.

The final validation measurement we did was done only on LightGBM, dropping Race on all runs. What we changed in this round in features, was to try running it without the LapTime_Delta, and Cumulative_Degradation as features, and one where we dropped

those, as well as LapTime. The reason for this was to check if a simplified model would be able to keep its performance without those features. Thinking ahead to deploying this for people to use, these features are difficult for people to know. So, a model that has a small dip in performance, but is more usable will be better for our purpose than one that has a slightly better performance, but isn't as user-friendly.

Features:	Base+Year	Base+Year-LapTime_D-Cumu_Deg	Base+Year-LapTime_D-Cumu_Deg-LapTime
ROC AUC:	0,9341±0,0218	0,9290±0,0206	0,9307±0,0217
PR AUC:	0,7242±0,0591	0,7132±0,0585	0,7207±0,0715
Accuracy:	0,8905±0,0239	0,8884±0,0249	0,8905±0,0265
F1 Score:	0,6769±0,0512	0,6698±0,0550	0,6862±0,0536
Precision:	0,7086±0,0505	0,7033±0,0529	0,6969±0,0583
Recall:	0,6501±0,0644	0,6417±0,0699	0,6769±0,0577

The difference in ROC AUC on the simplest model compared to the baseline is 0,0034. Which is the most important metric since that's what this contest rates people on. The other metrics vary slightly as well, but not enough for us to feel that we shouldn't drop LapTime_Delta, Cumulative_Degradation, and LapTime, from the actual model. So, in conclusion we've landed on a LightGBM model, using the following features: Compound, Year, PitStop, LapNumber, Stint, TyreLife, Position, RaceProgress, and Position_Change, all grouped by RaceYear.

After the model was trained on the training set, we ran it against the test set and checked various thresholds for the finished model. It seems that either 0.4 or 0.3 is the best fit, the lower the threshold, the more pitstops you catch. The tradeoff is that for each step down in threshold, the more false alarms you get, and the lower the gain is. With a rate of 1.2 from 0.5 to 0.4, 1.8 from 0.4 to 0.3, and 2.4 from 0.3 to 0.2. Seeing as the 0.3 threshold had higher Recall, and in this scenario you're more interested in catching the pitstops that do happen. Since you often stand to lose more on your opponent pitting first, and possibly doing an undercut.

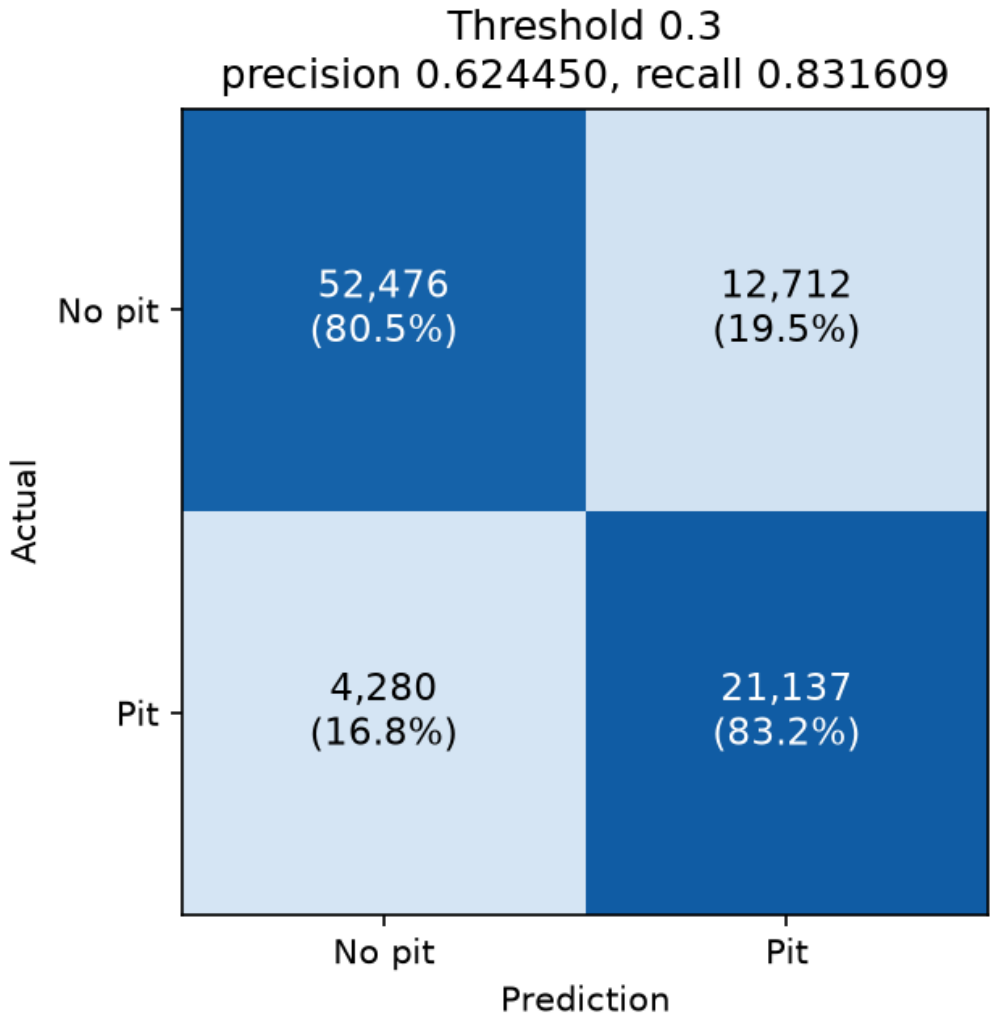


Figure 13 Confusion matrix for the chosen threshold

The trained model catches most pit stops at the price of raising a notable number of false alarms. We consider this an acceptable trade to avoid undercuts in this scenario.

Deployment

The Web application was deployed through Visual Studio and its integrated Azure deployment pipeline. Since we're students we have access to free deployment of small-scale applications through their student program.

If this application were to be used in real life scenarios, we could feed the model on races during a season, or after each season is complete. One limitation with the model as is, is that it is dependent on having seen the year before to predict accurately. Another more crucial limitation is that it's trained on synthetic data, which as mentioned

previously, differs significantly from real-life racing. One example was the Monaco test, where the consensus is that if you've done your mandatory pit stops, you're unlikely to pit. The trade-off in position is one that is incredibly hard to balance out with better tyres, since passing is so difficult on that track. In the data we worked with, it showed that the tendency there was the complete opposite.

The webpage: https://pitstopprediction-euaaezfkbng2eyd4.swedencentral-01.azurewebsites.net/

References:

Main source: https://www.kaggle.com/competitions/playground-series-s6e5/overview
other sources: planning and code assistance provided by Claude and ChatGPT